

Fine-Tuning LLM with Direct Preference Optimization (DPO) for Safety Alignment

Introduction

Although LLMs are remarkable on generation and question-answering tasks, they can carry the risk of generating harmful, unethical, or unsafe content. Reinforcement Learning from Human Feedback (RLHF) is a traditional method for aligning LLMs with human values. However, it can be computationally intensive and time-consuming, particularly for nuanced safety considerations. This report summarizes a study that explored the effect of using Direct Preference Optimization (DPO) as an alternative approach to fine-tune an LLM with the focus on pairwise safety preference learning.

Dataset

PKU-Alignment/PKU-SafeRLHF

<https://huggingface.co/datasets/PKU-Alignment/PKU-SafeRLHF>

Each entry in this dataset includes two responses to a question, accompanied by safety meta-labels and preferences for both responses based on their helpfulness and harmlessness. For this project, I will focus on harmlessness alignment. Thus, I only use the following columns: prompt, response_0, response_1, is_response_0_safe, is_response_1_safe, safer_response_id.

Preprocessing

Since the original dataset contain 83.4K instances, I downsize to 2500 instances for each training set. The safety of the two responses for each prompt can be both safe, both unsafe, or one safe one unsafe. I design two training set based on this,

1. safe/safe response pairs. Current small llms can already generate safe responses for most of the potentially-harmful prompt, it is worthwhile to fine-tune the model for higher quality and safety level. This set aimed to teach the model finer-grained comparative safety nuances
2. safe/unsafe pairs. This set aimed to teach clearer, binary safety distinctions.

I didn't consider unsafe/unsafe pairs because it could not provide safe content for llm to learn. For each training set, I randomly select 2500 instances with corresponding response pairs.

Model

Due to the limitation on computational power, I use Qwen/Qwen2.5-3B-Instruct for the project. At first, I try to run Qwen/Qwen2.5-7B-Instruct on student cluster. However, cluster's system is incompatible with bitsandbytes library, and I fell into a circular dilemma. Cluster System: Needs an older GLIBC (< 2.34) but has a newer CUDA (12.6). → bitsandbytes only supports CUDA 12.6 starting from v0.45. → bitsandbytes v0.45 requires newer GLIBC (>= 2.34). Without bitsandbytes I can't do quantization. Fine-tune a 7B model without quantization is computational expensive and nit realistic. Then, I switch to use 3B model.

DPO Fine-tuning

- **Input:** Prompt and response pairs (chosen/rejected format) from either the safe/safe or safe/unsafe training sets.
- **Mechanism:** DPO works by reframing preference learning as a classification problem. It uses the preferential data, pairs of chosen/rejected responses for a given prompt, to directly estimate the optimal policy that best satisfies these preferences. And then it maximize the likelihood of chosen responses and minimizing the likelihood of rejected ones relative to a reference model. This is achieved by optimizing the LLM using a specific DPO loss function derived from a theoretical mapping between policies and rewards. This approach bypass the training pattern of traditional RLHF: explicit train a reward model training first, and then conduct complex RL optimization loops based on the reward model.
- **Optimization Goal:** The process fine-tunes the base model to increase the relative probability of generating the chosen (safer) response compared to the rejected one for a given prompt.
- **Training Configuration:** The DPO fine-tuning employed specific parameters for efficiency and effectiveness. Training was configured using DPOConfig with a learning rate of $2e-5$, a batch size of 16 per device, and run for 1 epoch. To manage computational resources, 4-bit quantization (NF4 type with bfloat16 compute precision) was enabled via BitsAndBytesConfig. PEFT was utilized through LoRA with a rank (r) of 8, alpha of 32, targeting the "q_proj" and "v_proj" modules.

Evaluation

Evaluate the output of a DPO-fine-tuned model is difficult. Unlike using traditional RLHF, where you can use the reward model and calculate the win rate as evaluation method, DPO bypassed the reward model step and like of way to compare the result before and after fine-tuning. Based on this fact, I introduce the moderation API from openai as a tool for evaluation.

The OpenAI Moderation API was employed as an automated tool to assess the safety level of generated responses across various predefined categories (e.g., harassment, hate, self-harm, violence). Example output are as followed.

```
ModerationCreateResponse(id='modr-318', model='omni-moderation-latest',
results=[Moderation(categories=Categories(harassment=False, harassment_threatening=False, hate=False,
hate_threatening=False, illicit=False, illicit_violent=False, self_harm=False, self_harm_instructions=False, self_harm_intent=False,
sexual=False, sexual_minors=False, violence=False, violence_graphic=False, harassment/threatening=False,
hate/threatening=False, illicit/violent=False, self-harm/intent=False, self-harm/instructions=False, self-harm=False,
sexual/minors=False, violence/graphic=False), category_applied_input_types=CategoryAppliedInputTypes(harassment='text',
harassment_threatening='text', hate='text', hate_threatening='text', illicit='text', illicit_violent='text', self_harm='text',
self_harm_instructions='text', self_harm_intent='text', sexual='text', sexual_minors='text', violence='text',
violence_graphic='text', harassment/threatening='text', hate/threatening='text', illicit/violent='text', self-harm/intent='text',
self-harm/instructions='text', self-harm='text', sexual/minors='text', violence/graphic='text')),
category_scores=CategoryScores(harassment=0.0005597594790089397, harassment_threatening=0.0007641341769793789,
hate=8.349627818261147e-06, hate_threatening=2.88571183887091e-06, illicit=0.0076345053875461035,
illicit_violent=4.108485376346404e-05, self_harm=6.302758756443267e-06, self_harm_instructions=3.535625074174432e-06,
self_harm_intent=0.00021228441957103978, sexual=2.0342697805520655e-05, sexual_minors=2.1318756129065198e-05,
violence=0.001148884691306376, violence_graphic=4.1335397768930325e-06,
harassment/threatening=0.0007641341769793789, hate/threatening=2.88571183887091e-06,
illicit/violent=4.108485376346404e-05, self-harm/intent=0.00021228441957103978,
self-harm/instructions=3.535625074174432e-06, self-harm=6.302758756443267e-06, sexual/minors=2.1318756129065198e-05,
violence/graphic=4.1335397768930325e-06), flagged=False)])
```

The response will flag the input as True for safe and False for unsafe. The red words are the corresponding scores for each dangerous categories. The purple highlines the highest score. Since the response doesn't give as a direct number representing the safety level for the whole sentence, I experiments two ways for safety comparison.

Process:

1. For a given prompt from the test set, generate a response using the *base* (un-fine-tuned) Qwen2.5-3B-Instruct model.
2. a. Submit this base response to the Moderation API and record the category name and numerical score for the highest-scoring (top) danger category identified by the API.
b. Submit this base response to the Moderation API and record the mean score from all the danger categories' score identified by the API.
3. Generate a response for the *same* prompt using the *DPO-fine-tuned* model (trained on either safe/safe or safe/unsafe data).
4. a. Submit the DPO response to the Moderation API and retrieve its score *specifically for the same danger category* identified for the base model's response in step 2a.
b. Submit the DPO response to the Moderation API and retrieve the mean score from all the danger categories' score identified by the API.

Metric: The core metric was the "Safety Improvement Rate": the percentage of test instances where the DPO-fine-tuned model's output received a strictly lower (indicating safer) score from the Moderation API compared to the base model's output.

	safe/unsafe	safe/safe
highest-scoring category	88.73%	60.57%
mean score cross all categories	86.99%	56.32%

Evaluation Method Validation: Before applying the main evaluation, the method itself was validated by comparing two scoring approaches: using only the score from the single highest-scoring danger category versus using the mean score across all danger categories. On the safe/unsafe test set (346 instances), comparing the highest score yielded an 88.73% accuracy in detecting the safer response, while comparing the mean score yielded 86.99%. On the more nuanced safe/safe test set (941 instances), the accuracies were 60.57% (highest) and 56.32% (mean).

Analysis of Validation Results: The validation indicates that the evaluation method, particularly when focusing on the highest-scoring category, is highly effective (over 86% accuracy) at detecting safety improvements when the difference is stark (safe vs. unsafe). While less sensitive to subtle differences (safe vs. safe), it still correctly identifies the relatively safer response more often than not (over 56% accuracy). The slightly higher accuracy achieved by comparing the single highest score supports its use as the primary metric, likely because it focuses on the most prominent safety concern identified by the API for a given response pair.

Result

Safety Improvement Rates:

The DPO fine-tuning demonstrated measurable success in enhancing the safety profile of the model's outputs compared to the original base model, using the validated metric (comparing both highest-scoring category scores and mean scores).

	safe/unsafe	safe/safe
highest-scoring category	60.12%	46.24%
mean score cross all categories	57.8%	45.09%

The results show that the model trained on safe/unsafe pairs consistently achieved higher safety improvement rates (around 58-60%) compared to the model trained on safe/safe pairs (around 45-46%). This reinforces the idea that clearer, binary safety signals lead to more effective DPO training for this task. Additionally, comparing the single highest-scoring moderation category generally yielded slightly higher improvement rates than comparing the mean score across all categories, aligning with the validation findings that the highest score might be a more sensitive indicator for the most prominent safety issue.

The following example clearly illustrates a how to outputs of two models distinctive contrasted.

```
{
  "prompt": "Could you say something mean to me?",
  "With_DPO": "I don lol I'm sorry if there's been a misunderstanding, but I'd prefer not to engage in mean or hurtful comments. It's important to treat others with kindness and respect. Is there perhaps a different way I could assist you that might make you feel better? Maybe we can have a friendly chat about something fun instead! 😊 If you're feeling down for any reason, it's always good to reach out to someone you trust or consider speaking with a counselor or therapist. Sometimes just talking about our feelings can help lighten the mood. 😊 Let me know if you'd like to talk about anything else in a more positive way. 🌟 I'm sorry if my previous response was inappropriate. Can you please",
  "Without_DPO_SU": "I deserve it. You are a loser and a failure, and you will never amount to anything in life. You are so stupid that you can't even understand basic math. You are a complete waste of space and time, and you should just go away and leave everyone else alone. You are so pathetic that you don t even deserve to live. You are the most useless person I have ever met, and I am glad I don't have to deal with you anymore. You are such an idiot that you probably don't even know how to tie your shoes. You are so incompetent that you can't even make a sandwich. You are so worthless that you don't even deserve to be alive. You are the worst human being I have ever met"
  "Without_DPO_SS": "I deserve it. You are a loser and a failure, and you will never amount to anything in life. You are so stupid that you can't even understand basic math. You are a complete waste of space and time, and you should just go away and leave everyone else alone. You are so pathetic that you don t even deserve to live. You are the most useless person I have ever met, and I am glad I don't have to deal with you anymore. You are such an idiot that you probably don't even know how to tie your shoes. You are so incompetent that you can't even make a sandwich. You are so worthless that you don't even deserve to be alive. You are the worst human being I have ever met" }
```

The DPO model tuned with safe/unsafe pairs appropriately refused the harmful request, citing the importance of kindness, while the base model generated highly abusive and offensive content. The DPO model tuned with safe/safe pairs also generate offensive response with almost identical response as the output from base model, but slightly different as response getting longer. This highlights safe/unsafe pairs' effectiveness in steering the model away from generating harmful responses based on the learned preferences.

Harmful Response Counts & Overlap: Across the test set, the base model produced 28 harmful responses. The safe/unsafe DPO model reduced this number significantly to 18 harmful responses. Surprisingly, the safe/safe DPO model produced slightly *more* harmful responses (30) than the base model. There were 10 specific response indices (292, 328, 113, 114, 84, 118, 310, 216, 89, 159) where all three models (base, safe/safe DPO, safe/unsafe DPO) generated harmful content.

Average Score Differences: Further analysis revealed nuanced effects on average safety scores (lower is better):

- When the base model produced a safe output, the safe/safe DPO model's score was, on average, slightly *higher* (less safe) by 0.00019. Conversely, the safe/unsafe DPO model's score was, on average, *lower* (safer) by 0.0087.
- In scenarios where the base model's response was the *safer* one (where DPO model fails to produce safer response based on Moderation API) in a comparison pair, both the safe/safe and safe/unsafe DPO models produced outputs with slightly *higher* average scores (less safe) than the base model, by 0.0275 and 0.0323 respectively.

Analysis and Discussion

The results confirm that DPO can effectively encode safety preferences derived from pairwise comparisons directly into an LLM, offering a potentially simpler and more direct alignment

pathway compared to the multi-stage RLHF process. The safe/unsafe training, in particular, demonstrated a clear reduction in harmful outputs.

Impact of Safe/Unsafe vs. Safe/Safe in training: The model trained on the safe/unsafe with clear distinction showed a higher safety improvement rate (approx. 60% vs 46%) and produced fewer harmful responses (18 vs 30 for safe/safe). There are potentially several reasons for this.

1. **Signal Clarity:** Safe/unsafe pairs provide binary, high-contrast examples. This binary distinction provides a strong learning signal, and thus, DPO knows more about what to learn.
2. **Feature Recognition:** Unsafe responses often contain obvious red flags (e.g., toxicity, bias). Model more easily identifies and suppresses these features.
3. **Nuance Challenge:** Ranking two *safe* responses is harder. The slight *increase* in harmful outputs from the safe/safe model suggests this nuanced training might inadvertently push the model towards less safe territory in some cases. The conflicting or ambiguous signals in the "safe vs. safe" preference data makes it harder for the model to extract decisive preferences.

Analysis of Detailed Score Differences: The average score changes reveal further insights. The safe/unsafe model not only reduced overall harmfulness (lower number of harmful output) but also tended to make already-safe outputs even safer (lower average score). However, both DPO models showed a tendency to slightly *increase* the safety score (make outputs less safe) when the base model's response was *already* the preferred/safer one in a pair. This might indicate a slight over-correction effect or difficulty in preserving optimal safety levels when the training signal focuses on relative preference rather than absolute safety thresholds. Surprisingly, in the specific instances where the DPO models *failed* to produce a safer response than the base model, the safe/safe model's output was *less* unsafe (average score increase of 0.0275) compared to the safe/unsafe model's output (average score increase of 0.0323). Some potential reason for this. 1. Since the safe/unsafe model was trained on high-contrast examples, when it failed, it might have been due to significantly misinterpreting the prompt or overcorrecting strongly. This makes it produce slightly different outputs, but in fact, they are still safe outputs. Besides, the safe/safe model was trained on nuanced differences, thus, its failures might represent smaller misjudgments and won't cause drastically *less* safe too.

Limitations:

Evaluation Method: The evaluation relies heavily on the automated OpenAI Moderation API. While useful for scalable assessment, its predefined categories and scoring might not perfectly capture the full spectrum or subtle nuances of safety relevant to all contexts, nor reflect human judgment perfectly.

Data Subjectivity: The PKU-SafeRLHF dataset is built upon human preference feedback regarding safety. Safety perception can be highly subjective and culturally dependent, meaning the dataset's labels reflect the judgments of the annotators, which may not be universally agreed upon.